

Limiti del MUTEX

- Se un thread attende il verificarsi di una condizione su una risorsa condivisa con altri thread
- Con i soli mutex sarebbe necessario un ciclo del tipo:

```
while(1) {  
    lock(mutex);  
    if (<condizione sulla risorsa condivisa>)  
        break;  
    unlock(mutex);  
    ...  
}  
<sezione critica>;  
unlock(mutex);
```

Attesa, e verifica ciclica sulla var, finchè la condizione verificata non rompe il ciclo, quindi sblocco.

Limiti MUTEX

- I mutex sono come strumento di cooperazione risultano:
 - inefficienti
 - ineleganti
- e per risolvere elegantemente problemi di cooperazione servono altri strumenti
- Le variabili condizione sono un'implementazione delle variabili condizione teorizzate da Hoare

Una operazione attendi() su una variabile condizionale sospende un processo in una coda d'attesa per quella variabile condizionale, dando così via libera ad un nuovo processo che desidera entrare. L'operazione notifica() risveglia un processo sospeso sulla variabile condizionale; questo riprende l'esecuzione appena ha via libera

Variabili di Condizione

- strumenti di sincronizzazione tra thread che consentono di:
 - attendere passivamente il verificarsi di una condizione su una risorsa condivisa
 - segnalare il verificarsi di tale condizione
- la condizione interessa sempre e comunque una risorsa condivisa
- pertanto le variabili condizioni possono sempre associarsi al mutex della stessa per evitare corse critiche sul loro utilizzo

```
int pthread_cond_wait( pthread_cond_t *cond,  
                      pthread_mutex_t *mutex);
```

Variabili di Condizione

- Servono per attendere che una condizione si verifichi, escludendo race conditions (rendezvous tra thread)
- Utilizzata con i mutex: modificata dopo lock mutex e visibile dopo acquisizione del mutex
- Una condition variable è mantenuta in una struttura `pthread_cond_t`
- Tale struttura va allocata ed inizializzata
- Per inizializzare:
 - se la struttura è allocata staticamente:
`pthread_cond_t c = PTHREAD_COND_INITIALIZER`
 - se la struttura è allocata dinamicamente: chiamare `pthread_cond_init`

Inizializzare e distruggere una condition variable

```
int pthread_cond_init(    pthread_cond_t  *cond,  
                        const pthread_condattr_t *attr);
```

```
int pthread_cond_destroy(pthread_cond_t *cond);
```

- inizializza e distrugge una condition variable, rispettivamente
- Per la destroy non devono esistere thread in attesa
- restituiscono 0 se OK, un codice d'errore altrimenti
- attr può essere NULL (attributi di default)

Inizializzazione

```
#include <pthread.h>
extern void do_work ();

int thread_flag;
pthread_cond_t thread_flag_cv;
pthread_mutex_t thread_flag_mutex;

void initialize_flag() {
    // Inizializza mutex associato a variabile condizione
    pthread_mutex_init(&thread_flag_mutex, NULL);
    // Inizializza variabile condizione associata a flag
    pthread_cond_init(&thread_flag_cv, NULL);
    // Inizializza flag
    thread_flag = 0;
}
```

Usare una condition variable

- thread che aspetta una condizione

mutex_lock(m)

while (condizione falsa)

cond_wait(c, m)

fa' qualcosa

mutex_unlock(m)

- thread che rende la condizione vera

mutex_lock(m)

rendi la condizione vera

cond_broadcast(c)

mutex_unlock(m)

Attendere una condition variable

```
int pthread_cond_wait(pthread_cond_t *cond,  
pthread_mutex_t *mutex);
```

- attende che cond sia segnalata come vera
- restituisce 0 se OK, un codice d'errore altrimenti
- il mutex protegge la condizione
 - deve essere acquisito prima di chiamare cond_wait
 - durante l'attesa, cond_wait rilascia il mutex
 - finita l'attesa, cond_wait riprende il mutex

Attendere un variabile di condizione

```
#include <pthread.h>
```

```
int pthread_cond_wait(pthread_cond_t *cond,  
                      pthread_mutex_t *mutex);
```

- `cond` è una variabile condizione
 - `mutex` è un mutex associato alla variabile
1. al momento della chiamata il mutex deve essere bloccato
 2. rilascia il mutex, il thread chiamante rimane in attesa passiva di una segnalazione sulla variabile condizione
 3. nel momento di una segnalazione, la chiamata restituisce il controllo al thread chiamante, e questo rientra in competizione per acquisire il mutex
- restituisce 0 in caso di successo oppure un codice d'errore $\neq 0$

Esempio

```
...
/* Chiama do_work() mentre flag è settato, altrimenti si
   blocca in attesa che venga segnalato un cambiamento nel
   suo valore */
void* thread_function (void* thread_arg) {
    while (1) {
        // Attende segnale sulla variabile condizione
        pthread_mutex_lock(&thread_flag_mutex);
        while (!thread_flag)
            pthread_cond_wait(&thread_flag_cv, &thread_flag_mutex);
        pthread_mutex_unlock(&thread_flag_mutex);
        do_work ();          /* Fa qualcosa */
    }
    return NULL;
}
```

Inizializzare e distruggere una condition variable

```
int pthread_cond_signal(pthread_cond_t *cond);
```

```
int pthread_cond_broadcast(pthread_cond_t *cond);
```

- signal risveglia esattamente un thread in attesa su una condition variable
- broadcast risveglia tutti i thread in attesa su una condition variable
- restituiscono 0 se OK, un codice d'errore altrimenti
- attr può essere NULL (attributi di default)

Segnalazione

```
#include <pthread.h>
```

```
int pthread_cond_signal(pthread_cond_t *cond);
```

■ uno dei thread che sono in attesa sulla variabile condizione cond viene risvegliato

□ se più thread sono in attesa, ne viene scelto uno ed uno solo effettuando una scelta non deterministica

□ se non ci sono thread in attesa, non accade nulla

```
_// Setta flag a FLAG_VALUE
void set_thread_flag (int flag_value) {
    // Lock del mutex su flag
    pthread_mutex_lock (&thread_flag_mutex);
    // cambia il valore del flag
    thread_flag = FLAG_VALUE;
    // segnala a chi è in attesa che
    // il valore di flag è cambiato
    pthread_cond_signal (&thread_flag_cv);
    // unlock del mutex
    pthread_mutex_unlock (&thread_flag_mutex);
}
```

Timed Wait & Broadcast

```
#include <pthread.h>
int pthread_cond_timedwait( pthread_cond_t *cond,
                           pthread_mutex_t *mutex,
                           const struct timespec abstime);
```

- `cond` è una variabile condizione
- `mutex` è un mutex associato alla variabile
- `abstime` è la specifica di un tempo assoluto
- `pthread_cond_timedwait()` permette di restare in attesa fino all'istante specificato restituendo il codice di errore `ETIMEDOUT` al suo scadere
- restituisce 0 in caso di successo oppure un codice d'errore $\neq 0$

```
#include <pthread.h>
int pthread_cond_broadcast(pthread_cond_t *cond);
```

- causa la riparterza di tutti i thread che sono in attesa sulla variabile condizione `cond`
 - se non ci sono thread in attesa, non succede niente
- restituiscono 0 in caso di successo oppure un codice d'errore $\neq 0$

Esempio

```
pthread_mutex_t sem=PTHREAD_MUTEX_INITIALIZER;  
pthread_cond_t cond=PTHREAD_COND_INITIALIZER;
```

```
void *dec(void *arg){  
  
    while(1){  
        pthread_mutex_lock(&sem);  
        while (test.a==0)  
            pthread_cond_wait(&cond, &sem);  
  
        printf("CONSUMATORE 1 a=%d \n", test.a);  
        test.a--;  
        pthread_mutex_unlock(&sem);  
        nanosleep(&t2,NULL);  
    }  
}
```

Esempio

```
void *dec2(void *arg){  
  
    while(1){  
        pthread_mutex_lock(&sem);  
        while (test.a==0)  
            pthread_cond_wait(&cond, &sem);  
  
        printf("CONSUMATORE 2 a=%d, b=%d \n", test.a, test.b);  
        test.a--;  
        test.b--;  
        pthread_mutex_unlock(&sem);  
        nanosleep(&t2,NULL);  
    }  
}
```

Esempio

```
void *inc(void *arg){ // incremente a e b
```

```
    int j=0;
```

```
    while (j++<10){
```

```
        pthread_mutex_lock(&sem);
```

```
        test.a++;
```

```
        test.b++;
```

```
        printf("PRODUTTORE tid=%d a=%d b=%d\n", pthread_self(),test.a, test.b);
```

```
        pthread_cond_signal(&cond);
```

```
        pthread_mutex_unlock(&sem);
```

```
        nanosleep(&t2,NULL);
```

```
    }
```

```
    pthread_exit((void *)&test);
```

```
}
```

```
int main(void){
```

```
    pthread_t tid;
```

```
    pthread_create(&tid, NULL, inc, NULL);
```

```
    pthread_create(&tid, NULL, dec, NULL);
```

```
    pthread_create(&tid, NULL, dec2,NULL);
```

```
    sleep(5);
```


Esempio

PRODUTTORE tid=1077283760 a=1 b=1

CONSUMATORE 1 a=1

PRODUTTORE tid=1077283760 a=1 b=2

CONSUMATORE 1 a=1

PRODUTTORE tid=1077283760 a=1 b=3

CONSUMATORE 1 a=1

PRODUTTORE tid=1077283760 a=1 b=4

CONSUMATORE 2 a=1, b=4

PRODUTTORE tid=1077283760 a=1 b=4

CONSUMATORE 2 a=1, b=4

PRODUTTORE tid=1077283760 a=1 b=4

CONSUMATORE 2 a=1, b=4

PRODUTTORE tid=1077283760 a=1 b=4

CONSUMATORE 2 a=1, b=4

PRODUTTORE tid=1077283760 a=1 b=4

CONSUMATORE 2 a=1, b=4

PRODUTTORE tid=1077283760 a=1 b=4

CONSUMATORE 1 a=1

PRODUTTORE tid=1077283760 a=1 b=5

CONSUMATORE 1 a=1